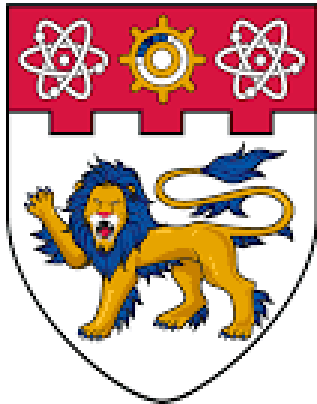




**NANYANG
TECHNOLOGICAL
UNIVERSITY**

SINGAPORE

System Requirement Specifications (SRS)

By SC3040 TEL5 Team 2

Dylan Quek Zhi En
Ang Wei Ming
Chi Seo Hyeon
Choo Kean Yee
Madelyn Cruz Tan
Lim Jun Howe
Tan Yong Huat

1. Table of Contents

1. Table of Contents	2
2. Problem Statement	3
3. Overview	3
3.1. Background	3
3.2. Overall Description	3
4. Investigation & Analysis Methodology	4
4.1. System Investigation	4
4.2. Analysis Methodology	4
4.2.1. Feasibility study and requirements elicitation	4
4.2.2. System analysis and requirements specification	4
4.2.3. Object-oriented design using UML	5
4.2.4. Prototyping	5
5. Constraints	6
5.1. Scalability	6
5.2. Data and Function Mapping	6
5.3. Proprietary hardware and software	6
5.4. Batch updates vs. (close) Real-time updates	6
5.5. Project Schedule	6
6. Operational Requirements	7
6.1. Application Services and Technical Support	7
6.2. Administration Features	7
6.3. System hardware failover and routine backup	7
6.4. Audit Trail	7
7. Functional Requirements	8
7.1. Registration	8
7.2. Login (Email & Password)	8
7.3. Citizen Features	8
7.3.1. View Issues	8
7.3.2. Submit Issue	9
7.3.3. Leave Review	9
7.3.4. Manage User Details	10
7.3.5. Chatbot	10
7.4. Administrator Features	10
7.4.1. Invite Government Account	10
7.5. Government Department Features	11
7.5.1. Update Status and Timeline	11
7.6. AI Categorisation & Duplicate Detection	11
7.6.1. Categorise and Assign Issue	11
7.6.2. Detect Duplicates	11
8. Input Requirements	13
8.1. User Identity	13
8.2. Issue Submission	13
8.3. Filters	13
9. Process Requirements	13
9.1. Database Transactions	13

9.2. Data integrity	13
9.3. Data validation	13
9.4. Performance	13
9.5. Data repository	13
10. Output Requirements	13
10.1. Transaction Summary and Confirmation	14
10.2. Exception Reports	14
10.3. Administrative Summaries	14
11. Hardware Requirements	14
11.1. Network	14
11.2. Client Devices	14
11.3. Hosting	14
12. Software Requirements	14
12.1. Client Operating Systems	14
12.2. Client Application	14
12.3. Network System	15
12.4. Backend/DB	15
12.5. External Services	15
12.6. Licenses	15
13. Deployment Requirements	15

2. Problem Statement

Singapore's Smart Nation and its 2.0 revamp have advanced digital services and encouraged citizen co-creation, but the government is aware of remaining gaps and new issues. Existing tools like OneService and the Kaki chatbot handle mostly municipal issues, and hackathons mirror community demand for more generic solutions. There is still no all-encompassing natural platform to receive and process a wider range of public issues, like accidents or defects in infrastructure, and connect citizens with a responsive, transparent form of engagement. Closing this gap requires a platform that is community-focused and inclusive and combines reporting and comment across agencies.

3. Overview

3.1. Background

In 2014, Singapore launched Smart Nation as a vision to use digital technology for the public good. Though developments have taken place, Smart Nation 2.0 detects voids and fresh opportunities for inclusive community solutions. Existing platforms, such as OneService and the Kaki chatbot, process municipal matters but are one-dimensional in nature and involve users needing to know whose agency is involved. Citizens are still unable to report large-scale matters such as accidents or infrastructure faults, and distinct systems lack transparency and feedback loops. ReportNow addresses this void by proposing a universal, citizen-centric platform that automatically channels reports to appropriate authorities, provides for real-time feedback and facilitates citizen engagement through transparent tracking and voting from society.

3.2. Overall Description

ReportNow provides citizens with an online system for registering, reporting issues in their town or neighbourhood council area, viewing issues on a map, and facilitating administrators in managing matters and users. The system will replace manual or separate channels for reporting and make effective issue management possible.

The system comprises a Next.js/React-built web client and backend with REST-style APIs and long-lived storage through a PostgreSQL database via Prisma ORM. Automatic issue tagging and duplicate verification are possible through AI functionality. The coverage includes the Citizen, Government and Admin UIs and their associated MVC services and models, e.g., role-based access control, authentication and AI services.

4. Investigation & Analysis Methodology

4.1. System Investigation

ReportNow allows users to register and log in using traditional credentials authentication. First-time users create an account record in a PostgreSQL database via Prisma ORM. After login, citizens submit issue reports with a title, description, category, GPS-based location and optional photos. Each report is stored and processed by AI services for categorisation and duplicate detection. Duplicate issues are linked to existing records to avoid redundancy. Valid reports appear on an interactive map within three minutes; administrators are notified to review and route issues to the correct government agency. Users can upvote reports to rank issues by community concern. Quality metrics, such as classification accuracy, duplicate detection latency, and issue resolution times, are logged for quality assurance.

4.2. Analysis Methodology

4.2.1. Feasibility study and requirements elicitation

Organise a development and implementation team of people with knowledge in user experience with municipal reporting platforms, AI technologies, and software quality assurance. Regular meetings and interviews with citizens, government agency representatives and developers of existing systems (e.g. OneService) will gather user needs and technical constraints. Feedback from course instructors will refine the scope. A Feasibility and Risk Assessment will evaluate alternative architectures (serverless vs. containerised deployments), AI model selection, regulatory compliance, data privacy considerations and the integration of OpenAI services. Findings will inform the final system requirements and risk management plans.

4.2.2. System analysis and requirements specification

4.2.2.1. Perform an analysis of the problem using object-oriented techniques

Object-oriented analysis through Unified Modelling Language (UML) will depict the actors (citizens, users from Government, administrators, super administrators and QA personnel), use cases (register, sign in, report issue, view issues, upvote, administer users, administer categories, search for duplicates, change status of issue and track quality metrics), and data flows. They will have an impact on the detailed System Requirement Specification covering functional requirements such as:

- 4.2.2.1.1. registration and sign in
- 4.2.2.1.2. AI-assisted classification
- 4.2.2.1.3. detection of duplicates in under five seconds
- 4.2.2.1.4. The map displays with ~10 m precision in locations
- 4.2.2.1.5. ordering by upvote
- 4.2.2.1.6. integration with an AI chatbot
- 4.2.2.1.7. administrative authority over users and categories
- 4.2.2.1.8. invitation links
- 4.2.2.1.9. logging and tracking risks and submission
- 4.2.2.1.10. tracking in quality metrics

Non-functional requirements consider performance, scalability, security, test coverage and regulatory compliance. The SRS will get progressively more detailed as more details emerge.

4.2.2.2. Scope and Limitations

Methodology encompasses:

- 4.2.2.2.1. Requirement analysis – Define system I/O, user requirements and functional and non-functional requirements, e.g., measures for security and privacy.
- 4.2.2.2.2. Data analysis – Plan data gathering (issue descriptions, user profiles, GPS coordinates, test results

and outputs from artificial intelligence), checking for validity, storing via Prisma ORM, manipulating (e.g. classification, joining of duplicates, amount of upvotes) and retrieving for quality measures and reporting.

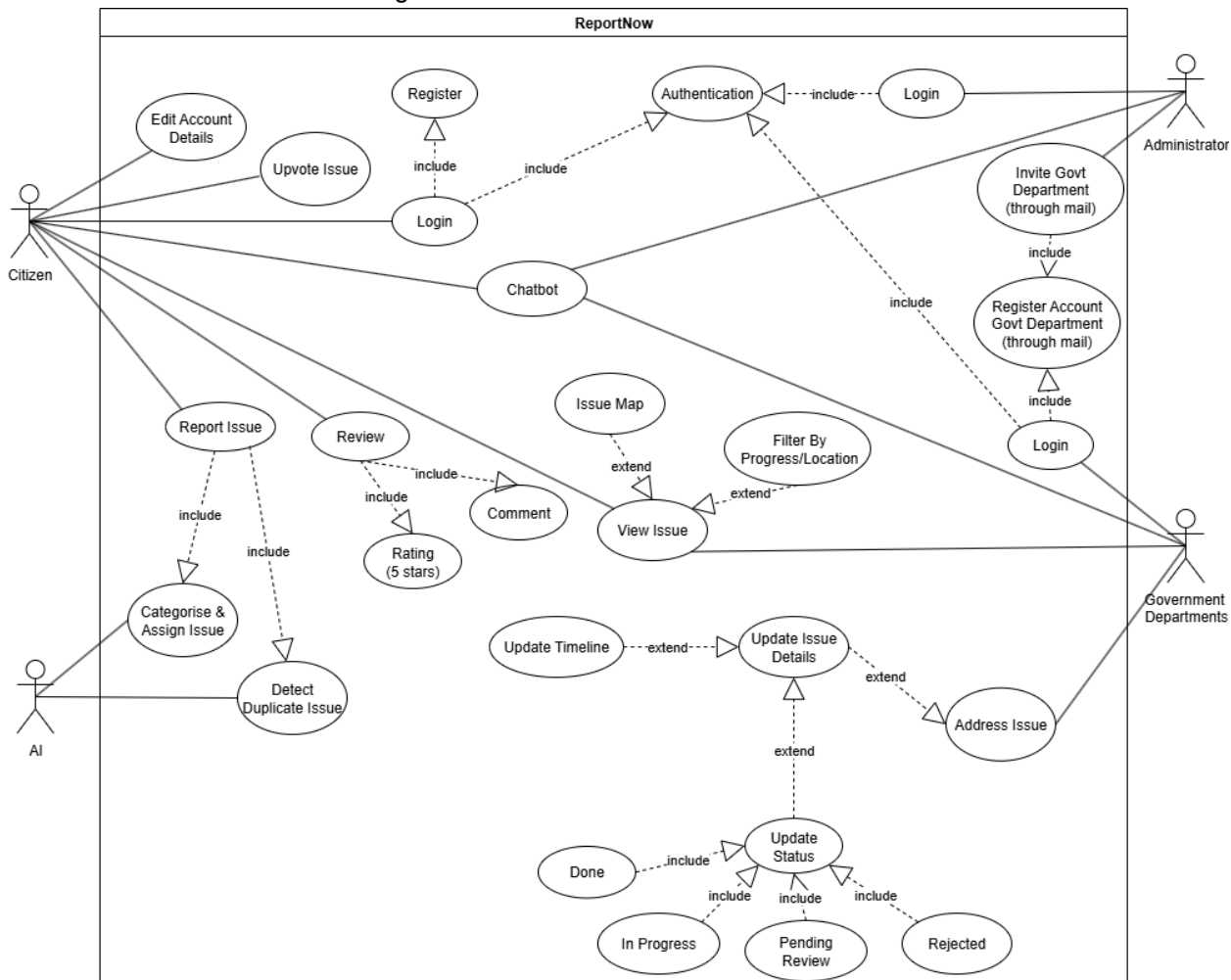
4.2.2.2.3. Process analysis – Analyse process flows and data (report submission, processing via AI, routing via agencies, alerting users to results, logging quality metrics) and system interfaces (OpenAI and government endpoints, REST APIs, web client).

4.2.2.2.4. Requirement analysis – Define system I/O, user requirements and functional and non-functional requirements, e.g., measures for security and privacy.

4.2.3. Object-oriented design using UML

A detailed object-oriented design for the ReportNow platform will be developed. UML diagrams will capture the key actors, use cases, and data flows. The primary actors in the system are as seen in the Overall Use Case Diagram below.

4.2.3.1. Overall Use Case Diagram



4.2.4. Prototyping

The Object-Oriented Rapid Prototyping (OORP) method will be applied to implement a limited but functional prototype for the ReportNow platform. The prototype demonstrates the essential features of the system, focusing on the citizen-facing reporting flow and the administrative management of issues.

The initial prototype will allow a citizen to register, log in, and submit an issue containing a title, description, GPS-based location, and optional photo. The issue will then be processed by the AI service for categorisation and duplicate detection, with results shown to the user. Submitted issues will appear on an interactive map within three minutes.

The prototype will also include an administrator interface to verify duplicates, change categories, and review submitted reports. This early version will not be a full deployment but will act as a working proof-of-concept to validate core requirements. It demonstrates the integration of the Next.js/React front-end, Node.js serverless backend, PostgreSQL database via Prisma ORM, and AI service calls for classification.

The prototype will be presented to the development team and quality assurance personnel for assessment, and feedback will be used to refine the design before broader implementation.

5. Constraints

5.1. Scalability

The platform must be able to scale horizontally to meet demands if there are more users. ReportNow shall be hosted in a cloud environment, which allows servers and databases to expand without local load. Performance expectations include handling simultaneous citizen submissions, running AI models for classification, and updating maps almost in real-time.

5.2. Data and Function Mapping

New functions added to the platform must integrate seamlessly with the existing architecture. For example, when introducing a new category for reports such as noise complaints, the update should propagate through the database schema, issue classification pipeline, and user interface without requiring manual rework of the frontend or extensive refactoring. This is achieved through Prisma ORM migrations and modularised API routes for backend-to-frontend integration.

5.3. Proprietary hardware and software

The system avoids reliance on proprietary government hardware or legacy systems. Instead, it is developed using open standards and widely adopted frameworks such as Next.js, React, PostgreSQL, and Prisma ORM. Proprietary dependencies, such as external AI APIs, are carefully selected for stability and compliance but may be substituted in the future if equivalent open solutions are available.

5.4. Batch updates vs. (close) Real-time updates

Reports submitted by citizens must be visible on the platform's map display within three minutes, ensuring the system functions close to real-time. Duplicate detection and report submission display is expected to complete within five seconds of submission. Unlike batch-based updates, which delay visibility, this approach enhances transparency and responsiveness. Continuous monitoring ensures that reporting latency stays within acceptable limits.

5.5. Project Schedule

The project timeline is in line with the lab deliverables and course timeline. A functional prototype is expected in early October 2025, as indicated within Lab 3 and 4's deliverables. Afterwards, development will focus on quality output, refining quality metrics, and preparing for a final demo by the end of the semester.

6. Operational Requirements

6.1. Application Services and Technical Support

Application developers will have access to the source code and issue tracking tools (GitHub and Trello) to address bugs and make improvements. Database administrators will monitor system health and address any potential data-caused downtime. After initial deployment, continuous integration pipelines will be implemented to ensure deployments can be rolled out or back without significant or any downtime.

6.2. Administration Features

System security and access levels will be defined through role-based access control. Citizens can access only their own accounts and submitted issues. Administrators can manage issues, categories, and users, while government staff can update only those issues assigned to their agencies. Super administrators retain global access to all data and dashboards.

6.3. System hardware failover and routine backup

Cloud hosting ensures that server instances are backed up and can fail over automatically to maintain availability. Daily backups of the PostgreSQL database are configured, with tested recovery procedures in case of corruption or downtime.

6.4. Audit Trail

Every critical transaction, such as issue submission, category change, duplicate resolution, or administrative action, is logged with timestamps and user identifiers. This audit trail ensures accountability and traceability for all stakeholders.

6.5. Help Desk Support

Provide user support for technical issues (login, performance, browser compatibility) and account recovery.

6.6. External-Integration Independence

The system should degrade gracefully if certain integrations (e.g., Cloudinary, OpenAI) are temporarily unavailable—core reporting, browsing, and status viewing remain operational; advanced functions queue or retry.

7. Functional Requirements

7.1. Registration

Description and Priority

Description: Allow users to register an Account with role-appropriate flows.

Priority: High

Stimulus/Response Sequences

Citizens open the “Register” page from the login screen. Government users receive an invitation email with a unique, time-limited link to a Government registration page. The system validates inputs and creates an Account with the appropriate role.

Functional Requirements

- 7.1.1. The app must allow users to access role-appropriate registration screens.
- 7.1.2. Citizens access the registration page through a “Register” link visible at the login screen.
- 7.1.3. Government users access the registration page through an invitation email sent by an administrator.
- 7.1.4. The app must display input fields for “Name”, “Email”, “Password”, and (optional) “Contact Number”.
 - 7.1.4.1. The app must display an error if “Register” is selected and a required field is empty or invalid (including invalid email, invalid contact number when provided).
- 7.1.5. The app must display a “Register” button once the inputs are entered.
- 7.1.6. The app must create an Account with the appropriate role when valid inputs are submitted.

7.2. Login (Email & Password)

Description and Priority

Description: Allow users to log into their accounts using email and password.

Priority: High

Stimulus/Response Sequences

Users enter their email and password and select “Login”. The system validates credentials. On success, a session is created and users are directed to the Issues screen; otherwise, an error is shown.

Functional Requirements

- 7.2.1. The app must display the option to log in with email and password.
- 7.2.2. The app must display input fields for “Email” and “Password”.
- 7.2.3. The app must display a “Login” button when inputs are present.
- 7.2.4. The app must validate user input data.
- 7.2.5. The app must log Users in successfully if the User has an existing ReportNow Account and the inputs are correct.
- 7.2.6. The app must direct Users to the Issues screen after successful login.
- 7.2.7. The app must display “Incorrect Email/Password” if the inputs are invalid or do not match an existing account.

7.3. Citizen Features

7.3.1. View Issues

Description and Priority

Description: System displays a list of reported issues, with options to view details, filter, and switch to a map view.

Priority: High

Stimulus/Response Sequences

After login, Users land on the Issues screen showing all reported issues. Users may filter, open details, or switch to the map

to see nearby issues.

Functional Requirements

- 7.3.1.1. The App must display a vertical list of reported issues.
- 7.3.1.2. The App must display the Issue's photo when available; otherwise, show an "Unavailable" photo.
- 7.3.1.3. The App must display the Issue's title, location, date/time of report, and current status.
- 7.3.1.4. The App should display the distance from the user's current location when possible.
- 7.3.1.5. The App must allow Users to open an Issue's detail page.
- 7.3.1.6. App must display a "Filter" button on the Issues screen
 - 7.3.1.6.1. The app must provide filters: Sort By (Newest, Oldest, Nearest, Random), Agency (each assignable department + "All Agencies"), Status (All, Pending, In Progress, Done).
 - 7.3.1.6.2. The app must provide "Reset All" and "Apply" actions; selecting "Apply" must update the list according to chosen filters.
- 7.3.1.7. The app must provide a Map tab
 - 7.3.1.7.1. The app must allow panning/zooming and display icons for issues within the current map view.
 - 7.3.1.7.2. The app must allow Users to select a map icon and navigate to that Issue's detail page.
- 7.3.1.8. On an Issue detail page, the App must show the Issue's photo (or Unavailable), title, location, date/time, status, timeline (status updates), reviews, and overall rating.
 - 7.3.1.8.1. If the current user is a Citizen, the App must display an option to leave a review and rating.

7.3.2. Submit Issue

Description and Priority

Description: Citizens can submit a new issue with details and optional attachments.

Priority: High

Stimulus/Response Sequences

From the Issues screen, Citizens open "Report" and provide a title, description, category, and location (GPS or map). Optional photos may be attached. The system validates inputs, runs AI categorisation and duplicate detection, and publishes the report.

Functional Requirements

- 7.3.2.1. The app must display input fields for "Title", "Description", "Category", and "Location".
- 7.3.2.2. The app must support GPS-based autofill for "Location" when permitted by the device; manual map selection must be allowed.
- 7.3.2.3. The app should support attaching one or more images.
- 7.3.2.4. The app must validate inputs and show errors for missing/invalid fields.
- 7.3.2.5. The app must create the Issue record after successful validation.
- 7.3.2.6. The app must invoke AI categorisation and duplicate detection on submission.
- 7.3.2.7. The app must display the newly submitted Issue on the map within three minutes of submission.
- 7.3.2.8. The app must flag potential duplicates within five seconds of submission and show the linkage on the Issue page.

7.3.3. Leave Review

Description and Priority

Description: Citizens can leave a star rating and comment on an Issue.

Priority: Medium

Stimulus/Response Sequences

A Citizen opens an Issue page, selects a rating and adds a comment, then submits. The review is saved; the Issue's average rating updates.

Functional Requirements

- 7.3.3.1. The app must display a five-star rating control and a numeric indicator.
- 7.3.3.2. Clicking a star must highlight that star and all to its left, and update the numeric value.
- 7.3.3.3. The app must display an input field for “Review”.
- 7.3.3.4. The app must display “Submit Review” when inputs are present.
- 7.3.3.5. After submission, the App must update the review list and average rating and show the new review on reload.

7.3.4. Manage User Details

Description and Priority

Description: Citizens can manage their account details stored in ReportNow.

Priority: Low

Stimulus/Response Sequences

After login, Citizens open the Account page from the top-right name, edit fields, then select “Update Profile”. The system validates and persists changes.

Functional Requirements

- 7.3.4.1. The app must display the account name at the top-right; selecting it must open the Account page.
- 7.3.4.2. The app must display editable fields for “Name” and “Email”, and (optional) “Contact Number”, pre-filled with current data.
- 7.3.4.3. The app must validate fields; show errors for empty “Name”, invalid email, or invalid contact number (when provided).
- 7.3.4.4. The app must update stored account details after successful validation.

7.3.5. Chatbot

Description and Priority

Description: Citizens can ask issue-related questions and receive Chatbot replies.

Priority: Low

Stimulus/Response Sequences

Citizens open the chat widget, enter questions, and receive generated replies; messages appear in session history.

Functional Requirements

- 7.3.5.1. The app must display a chat icon on all pages and open the chat display when the icon is selected.
- 7.3.5.2. The app must show chat history for the session, allow sending text messages, and display Chatbot replies.

7.4. Administrator Features

7.4.1. Invite Government Account

Description and Priority

Description: Admins can invite government departments to join ReportNow with a Government Department Account.

Priority: Low

Stimulus/Response Sequences

From the Admin Dashboard, an Admin enters an email and selects “Send Invitation”. The system generates a unique link valid for 7 days and emails it with instructions. The link directs recipients to the Government registration and expires after 7 days.

Functional Requirements

- 7.4.1.1. The App must display an input field for the invitee's email and a "Send Invitation" button.
- 7.4.1.2. The App must validate the email and show an error for empty/invalid input.
- 7.4.1.3. The App must generate a unique, time-limited registration link valid for 7 days.
- 7.4.1.4. The App must email the invitation link and instructions to the provided address.

7.5. Government Department Features

7.5.1. Update Status and Timeline

Description and Priority

Description: Government Departments can update status and timeline entries for assigned issues.

Priority: High

Stimulus/Response Sequences

From the Government Issues page, staff select an Issue, choose a new status, and add timeline updates. The system saves the changes and reflects them on the Issue page.

Functional Requirements

- 7.5.1.1. The App must provide an "Edit" control for Issue status.
- 7.5.1.2. The App must present a dropdown with: Pending, In Progress, Done, Rejected; "Cancel" must discard changes; "Update" must persist the selection.
- 7.5.1.3. The App must allow staff to enter a new timeline update and add it to the Issue.
- 7.5.1.4. The App must persist status/timeline updates and show updated details thereafter.

7.6. AI Categorisation & Duplicate Detection

7.6.1. Categorise and Assign Issue

Description and Priority

Description: The AI module processes issue descriptions, categorises the issue, and suggests a relevant department; uncertain cases are flagged for manual review.

Priority: High

Stimulus/Response Sequences

When a new Issue is created, the AI analyses the description. The system updates the category and suggested department; if confidence is low, it flags the Issue for Admin review.

Functional Requirements

- 7.6.1.1. The system must send new Issue descriptions to the AI module for processing.
- 7.6.1.2. The AI module must analyse text (NLP), determine an issue type, and update the Issue category in the database.
- 7.6.1.3. The AI module should suggest the most relevant department.
- 7.6.1.4. The system must flag the Issue for manual review if AI confidence is below a defined threshold.
- 7.6.1.5. The system must allow admin-initiated AI updates to improve accuracy and allow admins to override AI-assigned values.

7.6.2. Detect Duplicates

Description and Priority

Description: The system detects potential duplicates to reduce redundancy and surface a canonical Issue.

Priority: High

Stimulus/Response Sequences

Upon submission, the system evaluates similarity to existing Issues and links probable duplicates into a group; the Issue page

shows duplicate linkage and the canonical record.

Functional Requirements

- 7.6.2.1. The system must evaluate new Issues for similarity within five seconds of submission.
- 7.6.2.2. The system must associate potential duplicates via a duplicate group identifier.
- 7.6.2.3. The system must display duplicate linkage on the Issue page and in admin views.
- 7.6.2.4. Admins must be able to confirm or dismiss duplicates and change the canonical Issue for the group.

8. Input Requirements

8.1. User Identity

Email (unique), password/Singpass tokens when used.

8.2. Issue Submission

Title, description, photos (≤ 100 MB), location (GPS/map), time.

8.3. Filters

Sort By (Newest/Oldest/Nearest/Random), Agency, Status (All/Pending/In Progress/Done).

8.4. Status/Timeline Updates

Status dropdown; free-text timeline entry.

8.5. Reviews

Star rating (0–5), comment text.

9. Process Requirements

The following are among the inherent requirements that the system must be able to handle.

9.1. Database Transactions

Create/Read/Update for Issues, Users, Departments, Timelines, Reviews on PostgreSQL with ACID semantics.

9.2. Data integrity

Commit only transactions that are completed and/or roll back unfinished or time-out transactions.

9.3. Data validation

Strict client/server validation for required fields, email/contact formats, media size, and permissible states.

9.4. Performance

Meet targets—map ≤ 5 s, report visible ≤ 3 s, profile updates ≤ 0.5 s, crash recovery ≤ 5 min, responsive up to ~ 500 concurrent users, handle concurrent use of the system on a 24/7 basis.

9.5. Data repository

PostgreSQL as the primary system of record; media in Cloudinary with references in the DB.

10. Output Requirements

10.1. Transaction Summary and Confirmation

Success/failure toasts/dialogues after submission, updates, and profile changes; email notifications when applicable.

10.2. Exception Reports

Highlight AI low-confidence classifications, invalid inputs, duplicate/suspicious reports for admin attention.

10.3. Administrative Summaries

Basic counts/roll-ups (e.g., by status, by department, resolution times) to help monitor throughput and service levels.

11. Hardware Requirements

11.1. Network

Reliable internet connectivity (wired/wireless)

11.2. Client Devices

Smartphones, tablets, laptops, desktops with GPS (for location) and ≥ 2 GB RAM

11.3. Hosting

Cloud-hosted application runtime and managed database; storage for images; UPS/redundancy handled by provider

12. Software Requirements

12.1. Client Operating Systems

- ☐ Linux
- ☐ macOS
- ☐ Windows
- ☐ Android (≥ 8.0)
- ☐ iOS (≥ 12)
- ☐ Browsers: latest

12.2. Client Application

Modern, standards-compliant browser (HTML5/CSS3/JS):

- ☐ Netscape
- ☐ IE
- ☐ Opera
- ☐ Chrome

12.3. Network System

Network software and protocols for systems to communicate:

- ☐ TCP/IP
- ☐ HTTP
- ☐ HTTPS
- ☐ TLS

12.4. Backend/DB

- ☐ Next.js/Node.js
- ☐ PostgreSQL (Prisma ORM)
- ☐ NextAuth

12.5. External Services

- ☐ Cloudinary
- ☐ OpenStreetMap (maps)
- ☐ NodeMailer (email)
- ☐ OpenAI (AI classification & chatbot)

12.6. Licenses

Valid licenses are required to run software from third-party vendors:

- ☐ To use application development tools
- ☐ To use web server, application server and database software in development, test and production modes

13. Deployment Requirements

